

AWS 실습 핸즈온

Amazon Bedrock 으로 만드는 RAG 사내 지식봇

콘솔(UI) 기반 · 코드 작성 없음

총 3 시간 (실습 콘텐츠 약 2 시간 20 분)

대상: AWS 입문자 (클라우드 경험자 기준)

리전: 미국 동부 (버지니아 북부) us-east-1

작성 기준일: 2026-06-30

목차

0. 시작하기 전에	4
이 실습에서 만드는 것.....	4
학습 목표.....	4
전체 일정 (3 시간).....	4
사전 준비물	4
1. RAG 개념 이해 (강의 20 분).....	6
RAG 란 무엇인가	6
RAG vs Fine-tuning	6
이 실습의 아키텍처	6
모듈 1. Bedrock 모델 액세스 활성화 (15 분)	7
단계.....	7
모듈 2. S3 버킷 생성 & 문서 업로드 (20 분).....	10
2-1. 버킷 만들기.....	10
2-2. 문서 업로드.....	11
모듈 3. Knowledge Base 생성 (30 분)	13
단계.....	13
모듈 4. 데이터 동기화 & 지식봇 테스트 (30 분).....	16
4-1. 문서 색인(Sync).....	16
4-2. 콘솔에서 질의응답 테스트	16
모듈 5. 심화 실험 (20 분).....	18
실험 1. 검색만 vs 생성까지.....	18
실험 2. 생성 모델 바꿔보기	18
실험 3. 검색 청크 수 조정	15
실험 4. 청킹 전략 비교 (새 KB 필요).....	18
모듈 6. 리소스 정리 (15 분).....	20
삭제 순서.....	20
부록 A. 트러블슈팅	21
왜 노트북(코드) 방식은 더 어려웠나	21
부록 B. Azure ↔ AWS 서비스 대응표	22
부록 C. 전체 완료 체크리스트	23

실습 중	23
실습 후 (비용 방지).....	23
부록 D. 제공 샘플 문서	23

0. 시작하기 전에

이 실습에서 만드는 것

회사의 문서(교육과정 안내)를 업로드하면, 그 문서 내용을 근거로 질문에 답하는 **사내 지식봇**을 만듭니다. 직원이 “데이터 엔지니어링 과목 선수과목이 뭐야?”라고 물으면, 봇이 업로드된 문서를 검색해 정확한 답을 돌려줍니다. 이 방식을 **RAG(검색 증강 생성)**라고 부릅니다.

핵심은 **코드를 한 줄도 작성하지 않는다**는 점입니다. 모든 작업을 AWS 웹 콘솔(화면 클릭)으로 진행합니다. 복잡한 권한 설정이나 벡터 데이터베이스 구성은 Bedrock 콘솔이 **자동으로** 처리해 줍니다.

학습 목표

- 생성형 AI 에서 RAG 가 왜 필요한지, Fine-tuning 과 어떻게 다른지 설명할 수 있다.
- Amazon Bedrock 에서 사용할 AI 모델의 액세스를 활성화할 수 있다.
- S3 에 문서를 업로드하고, 이를 Knowledge Base 의 데이터 소스로 연결할 수 있다.
- Knowledge Base 를 생성하고 문서를 색인(동기화)한 뒤, 콘솔에서 직접 질의응답을 테스트할 수 있다.
- 실습 후 리소스를 안전하게 삭제해 불필요한 비용을 막을 수 있다.

전체 일정 (3 시간)

순서	내용	방식	소요
도입	RAG 개념 강의 + 실습 환경 점검	강의	20 분
모듈 1	Bedrock 모델 액세스 활성화	실습	15 분
모듈 2	S3 버킷 생성 & 문서 업로드	실습	20 분
모듈 3	Knowledge Base 생성 (자동 구성)	실습	30 분
—	휴식	—	10 분
모듈 4	데이터 동기화 & 지식봇 테스트	실습	30 분
모듈 5	심화 실험 (프롬프트·검색·청킹)	실습	20 분
모듈 6	리소스 정리 (비용 방지)	실습	-
마무리	Q&A, 실무 연결, 회고	토론	20 분

사전 준비물

- **AWS 콘솔 로그인 계정** — 강사가 배포한 IAM 사용자 계정으로 로그인합니다. (루트 계정 사용 금지)
- **권장 브라우저** — Chrome 또는 Edge 최신 버전
- **리전 설정** — 콘솔 오른쪽 위 리전이 “미국 동부(버지니아 북부) us-east-1”인지 확인
- **샘플 문서 3 개** — ai_basic.txt, cloud_computing.txt, data_engineering.txt (제공됨)

Azure 사용자를 위한 안내

Azure 에서 “Azure AI Search + Azure OpenAI On Your Data”를 써보셨다면, 이 실습은 그 AWS 버전입니다. **Azure AI Search $\approx$ OpenSearch(벡터 DB), Azure OpenAI $\approx$ Amazon Bedrock** 이라고 생각하면 됩니다. 자세한 대응표는 부록 B 에 있습니다.

1. RAG 개념 이해 (강의 20 분)

RAG 란 무엇인가

RAG 는 Retrieval-Augmented Generation(검색 증강 생성)의 약자입니다. LLM 이 답을 **지어내지(환각)** 않도록, 답하기 전에 **신뢰할 수 있는 문서를 먼저 검색**해서 그 내용을 근거로 답하게 하는 기법입니다. 동작 순서는 다음과 같습니다.

- 1. **사용자 질문** 입력 (예: "클라우드 컴퓨팅 과목 평가 비중은?")
- 2. **검색(Retrieval)** — 질문을 벡터로 바꿔 벡터 DB 에서 의미가 비슷한 문서 조각을 찾음
- 3. **증강(Augmented)** — 찾은 문서 조각을 질문과 합쳐 "확장 프롬프트"를 구성
- 4. **생성(Generation)** — LLM 이 그 근거를 바탕으로 출처 있는 답변을 생성

RAG vs Fine-tuning

항목	RAG	Fine-tuning
데이터 최신성	문서만 바꾸면 즉시 반영	재학습 필요
비용	검색 비용 위주, 저렴	대규모 GPU 학습 비용
환각 위험	출처 기반이라 낮음	여전히 발생 가능
적합한 경우	문서 Q&A, 사내 지식 검색	특정 말투·도메인 문체 학습
구현 난이도	상대적으로 쉬움	높음

이 실습의 아키텍처

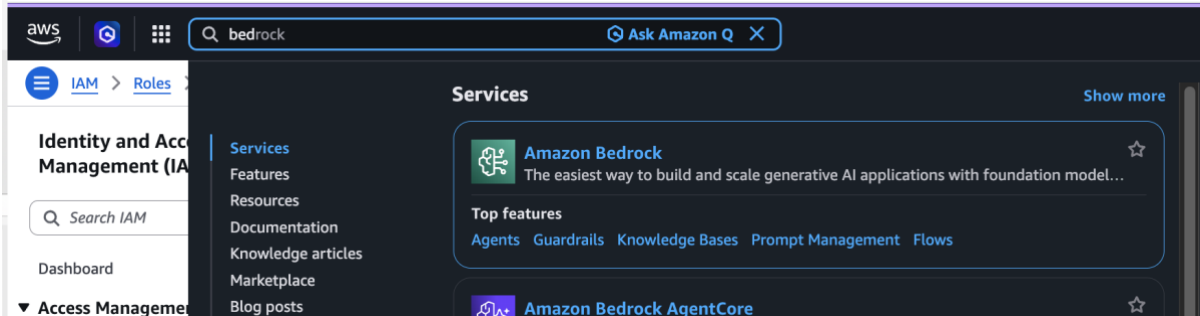


모듈 1. Bedrock 모델 액세스 활성화 (15 분)

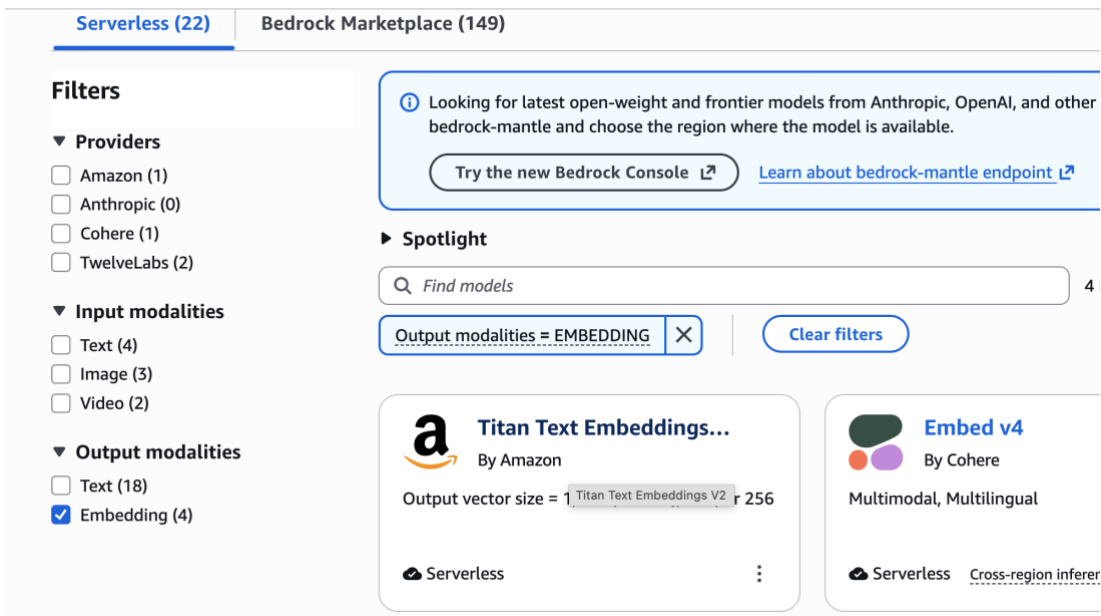
Bedrock 은 보안을 위해 처음에는 모든 AI 모델이 잠겨 있습니다. 사용할 모델 두 가지의 액세스를 먼저 켭니다. 임베딩 모델(문서를 벡터로 변환)과 생성 모델(답변 작성)입니다.

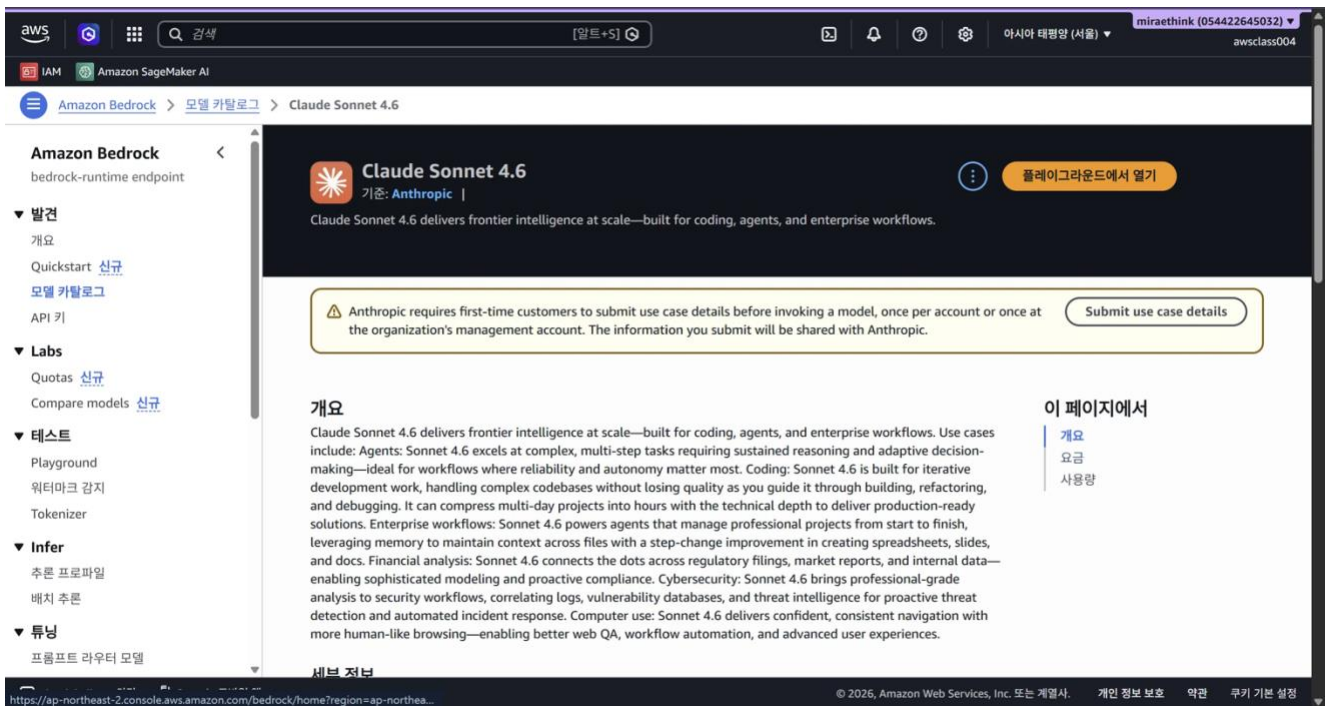
단계

1. AWS 콘솔 오른쪽 위에서 리전이 **US East (N. Virginia) us-east-1** 인지 확인합니다.
2. 상단 검색창에 **Bedrock** 을 입력하고 **Amazon Bedrock** 콘솔로 들어갑니다.



3. **View Model catalog** 를 클릭합니다.
4. 다음 모델을 선택합니다.
 - Amazon → **Titan Text Embeddings V2** (문서 임베딩용)
 - Anthropic → **Claude Haiku 4.5** (답변 생성용)





사용 사례(use case)버튼이 보이면, 이를 작성합니다.



Claude Haiku 4.5
 By: Anthropic |


[Open in playground](#)

Claude Haiku 4.5 delivers near-frontier performance for a wide range of use cases, and stands out as one of the best coding and agent models—with the right speed and cost to power free products and high-volume user experiences.

Overview
 Claude Haiku 4.5 delivers near-frontier performance for a wide range of use cases, and stands out as one of the best coding and agent models—with the right speed and cost to power free products and high-volume user experiences. Use cases: Powering free tier user experiences: Claude Haiku 4.5 delivers near-frontier performance at a cost and speed that makes powering free agent products and agentic use cases economically viable at scale. Real-time experiences: Claude Haiku 4.5's speed is ideal for real-time applications like customer service agents and chatbots where response time is critical. Coding sub-agents: Use Claude Haiku 4.5 to power sub-agents, enabling multi-agent systems that tackle complex refactors, migrations, and large feature builds with quality and speed. Financial sub-agents: Use Claude Haiku 4.5 to monitor thousands of data streams—tracking regulatory changes, market signals, and portfolio risks to preemptively adapt compliance and trading systems at previously impossible scales. Research sub-agents: Perform parallel analyses across multiple data sources while maintaining fast response times. Ideal for rapid business intelligence, competitive analysis, and real-time decision support. Business tasks: Claude Haiku 4.5 is capable of producing and editing office files like slides, documents, and spreadsheets. It also better supports strategy and campaign planning, business analysis and brainstorming.

On this page
[Overview](#)
[Pricing](#)
[Usage](#)

⚠ 주의

Anthropic 모델은 신청 시 **사용 사례(use case)** 입력을 요구할 수 있습니다. "Internal document Q&A for a university lab" 정도로 간단히 적으면 됩니다. 모델이 목록에 안 보이면 리전이 us-east-1 이 맞는지 다시 확인하세요.

✅ 체크포인트

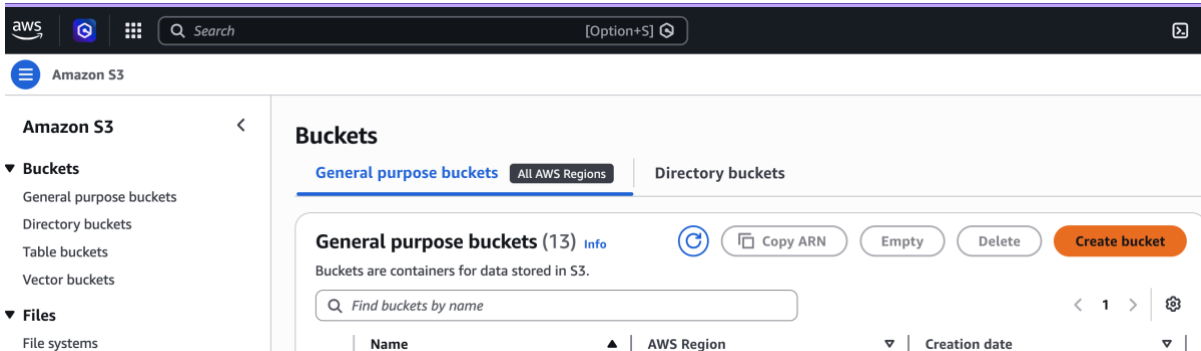
- ☐ Titan Text Embeddings V2 = Access granted
- ☐ Claude (Sonnet 또는 Haiku) = Access granted

모듈 2. S3 버킷 생성 & 문서 업로드 (20 분)

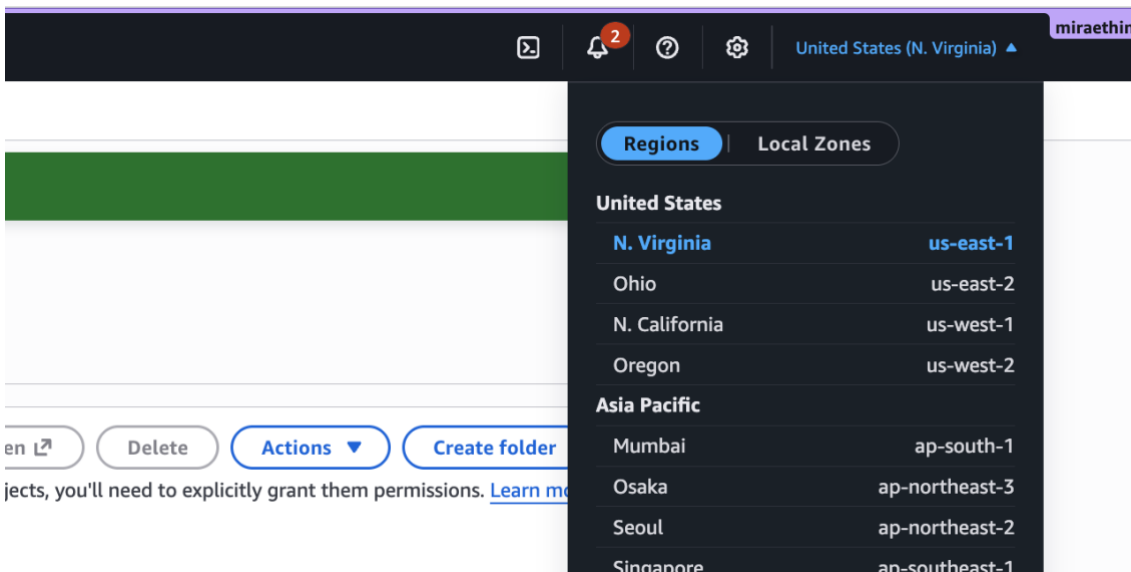
지식봇이 읽을 **원본 문서 저장소**를 만듭니다. S3 는 AWS 의 파일 저장 서비스로, Azure 의 Blob Storage 에 해당합니다.

2-1. 버킷 만들기

1. 상단 검색창에 **S3** 를 입력해 S3 콘솔로 이동합니다.
2. **Create bucket** 을 클릭합니다.



3. 버킷 이름을 **전 세계에서 고유한 값**으로 입력합니다. 예: **bedrock-kb-1ab-본인이름-0630**
4. Region 이 **us-east-1** 인지 확인합니다.



5. 나머지는 기본값(퍼블릭 액세스 차단 유지)으로 두고 맨 아래 **Create bucket** 을 누릅니다.

💡 버킷 이름 규칙

S3 버킷 이름은 **전 세계에서 단 하나**여야 합니다. 소문자, 숫자, 하이픈(-)만 쓰고 공백·대문자·언더스코어는 안 됩니다. "Bucket already exists" 오류가 나면 뒤에 숫자를 더 붙이세요.

참고:

1. Global namespace (글로벌 네임스페이스)

기존에 우리가 흔히 알던 전통적인 S3 버킷 생성 방식입니다.

- **이름 고유성 범위: 전 세계 모든 AWS 계정**
- **특징:** 내가 만든 버킷 이름은 전 세계의 다른 어떤 AWS 사용자도 사용할 수 없습니다.
 - * **단점 (단어 선점 경쟁):** test-bucket, my-data, backup 같은 직관적이고 쉬운 이름은 이미 전 세계 누군가가 선점했기 때문에 사용할 수 없습니다. 이 때문에 버킷 이름을 지을 때마다 뒤에 무작위 숫자나 회사명을 붙여야 하는 번거로움이 있었습니다.
- **URL 형태:** 기본적으로 `https://[버킷이름].s3.[리전].amazonaws.com` 형태로 전역에서 접근 가능한 고유 주소를 가집니다.

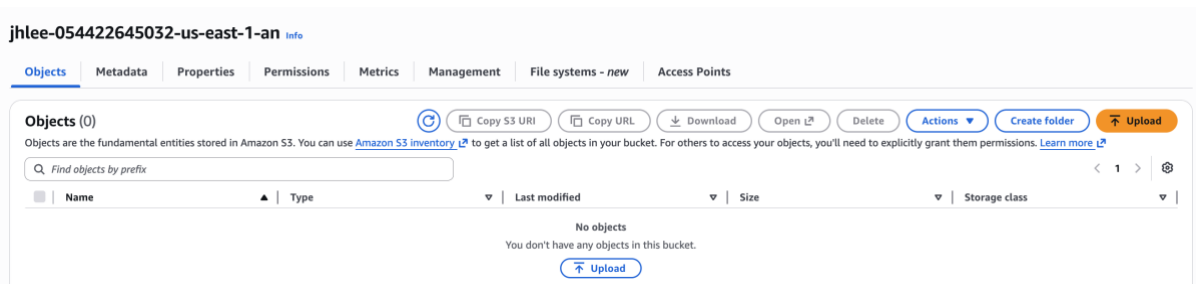
2. Account Regional namespace (계정 리전별 네임스페이스)

최근 AWS 에 새로 도입되어 권장(Recommended)되는 방식입니다.

- **이름 고유성 범위: 내 AWS 계정 + 특정 리전(Region) 내부**
- **특징:** 버킷 이름의 고유성이 '내 계정'과 '선택한 리전' 안에서만 보장되면 됩니다. 즉, 다른 AWS 계정에서 이미 사용 중인 이름이더라도, 내 계정에서는 똑같은 이름으로 버킷을 생성할 수 있습니다.
- **장점:** 더 이상 전 세계 사용자와 이름 짓기 경쟁을 할 필요가 없습니다. 내 계정 안에서 직관적으로 raw-data, processed-data 같은 깔끔한 이름을 리전별로 자유롭게 사용할 수 있습니다.
- **보안성:** 외부에서 우연히 내 버킷 이름을 추측하여 접근하려는 시도를 원천 차단하기 유리하며, 계정 경계 내에서 자원을 관리하기 때문에 거버넌스 측면에서 더 안전합니다.

2-2. 문서 업로드

1. 방금 만든 버킷 이름을 클릭해 들어갑니다.



2. **Create folder** 로 curriculum 폴더를 만듭니다. (선택이지만 권장)
3. curriculum 폴더로 들어가 **Upload** → **Add files** 를 누릅니다.

4. 제공된 샘플 문서 3 개를 선택합니다.

- ai_basic.txt
- cloud_computing.txt
- data_engineering.txt

5. **Upload** 을 눌러 업로드하고, 3 개 파일이 모두 보이는지 확인합니다.

✔ Upload succeeded
For more information, see the [Files and folders](#) table.

Upload: status

Close

ⓘ After you navigate away from this page, the following information is no longer available.

Summary

Destination
s3://jhlee-054422645032-us-east-1-an/curriculum/

Succeeded
✔ 3 files, 2.7 KB (100.00%)

Failed
⋯ 0 files, 0 B (0%)

Files and folders

Configuration

Files and folders (3 total, 2.7 KB)

< 1 >

Name	Folder	Type	S
ai_basic.txt	-	text/plain	9
cloud_computing.txt	-	text/plain	8
data_engineering.txt	-	text/plain	9

✔ 체크포인트

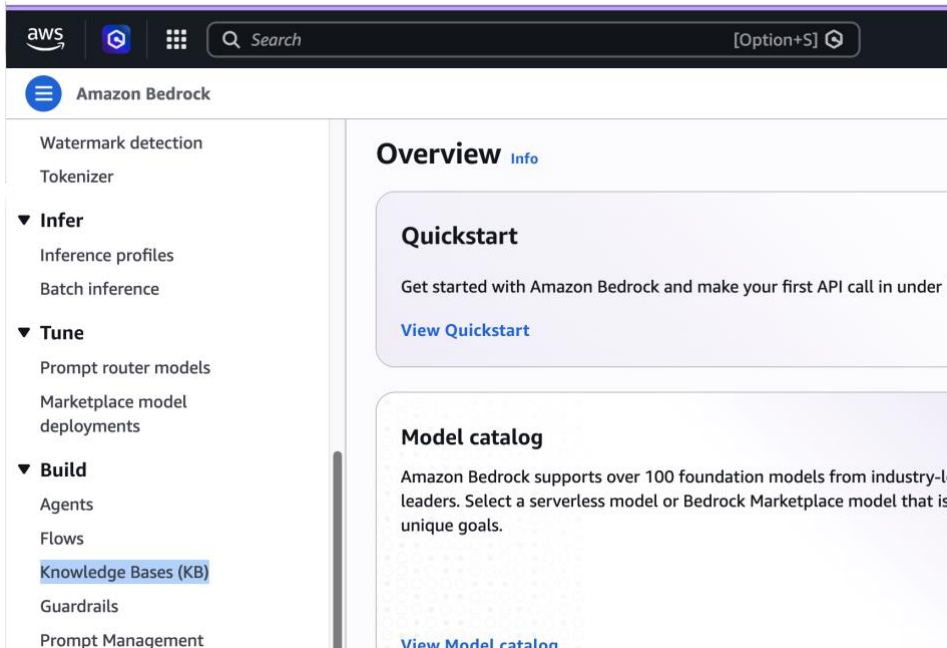
- ☐ 버킷이 us-east-1 에 생성됨
- ☐ curriculum/ 아래에 .txt 문서 3 개 업로드 완료

모듈 3. Knowledge Base 생성 (30 분)

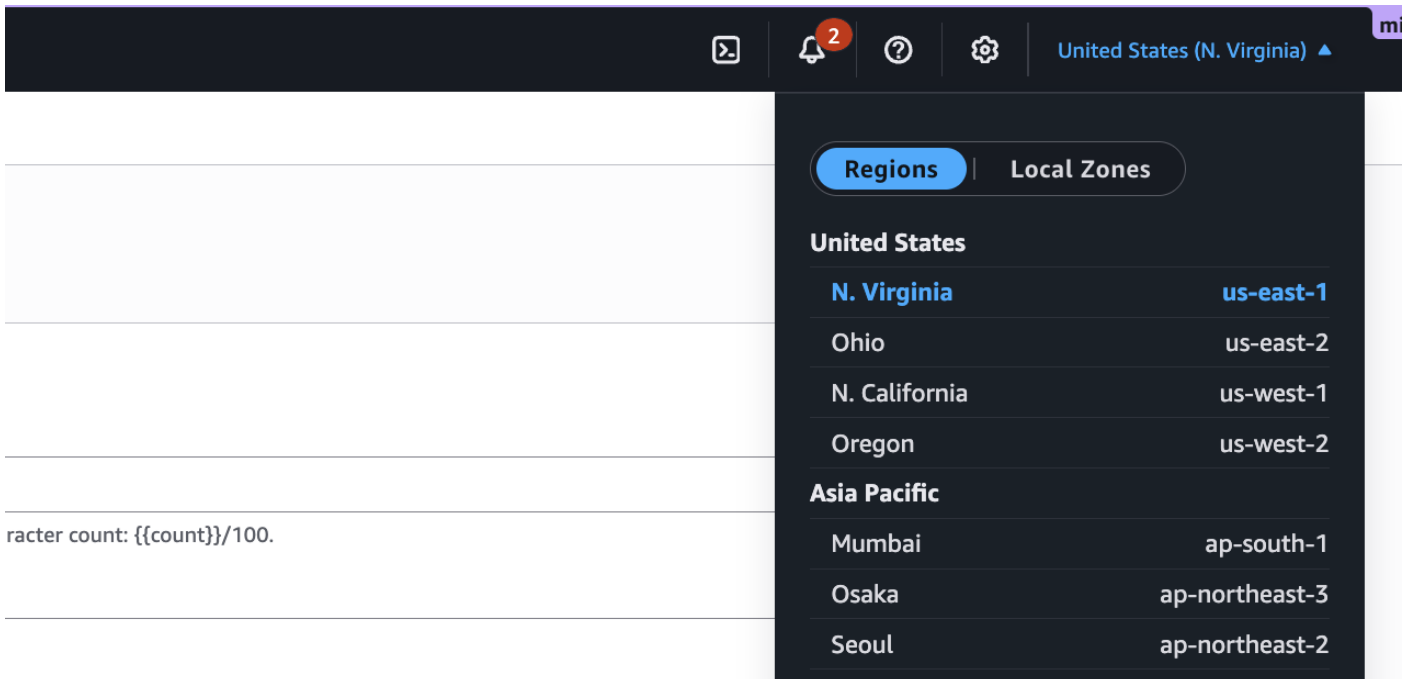
이번 모듈이 핵심입니다. Bedrock 이 벡터 DB·IAM 역할·인덱스를 모두 자동 생성하도록 “Quick create” 옵션으로 진행합니다.

단계

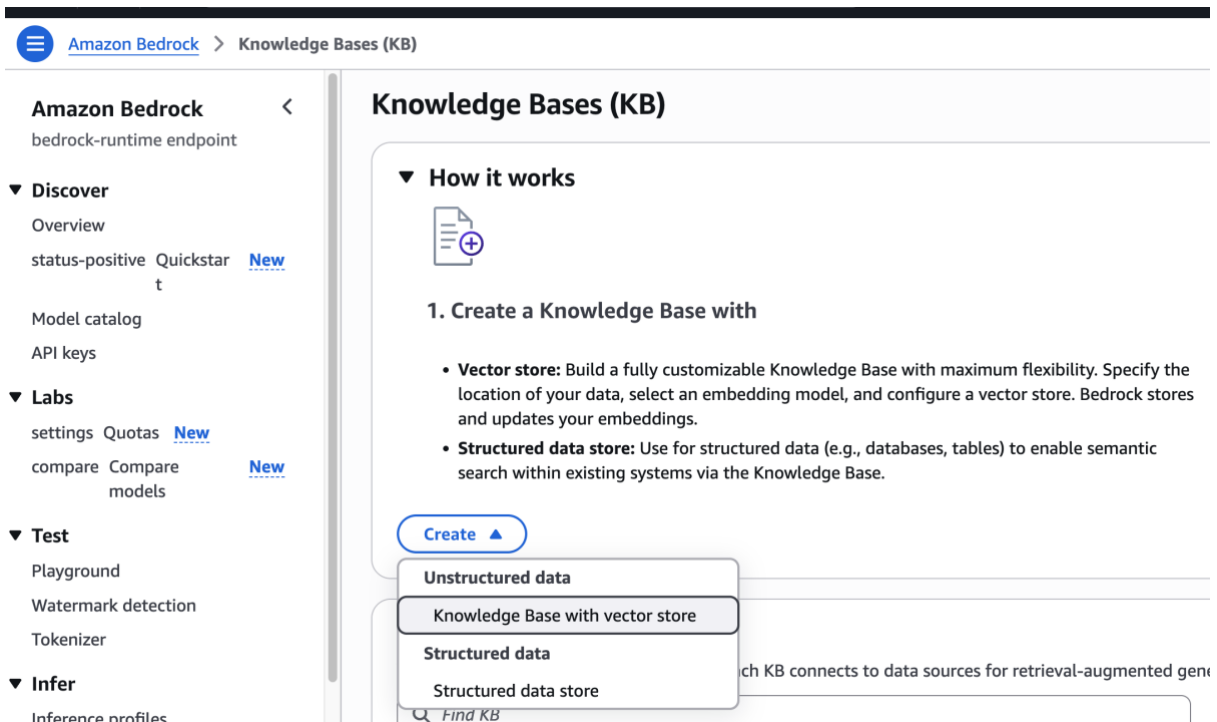
1. Amazon Bedrock 콘솔 → 왼쪽 메뉴 **Knowledge Bases** 로 이동합니다.



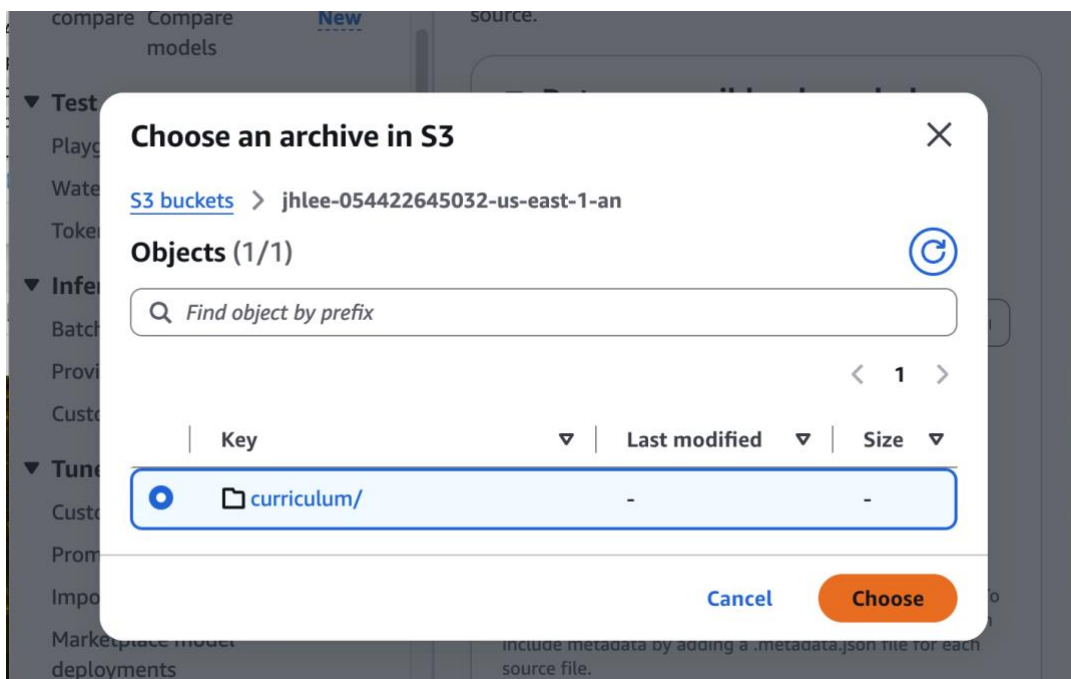
여기서도 region 을 맞춰줍니다.



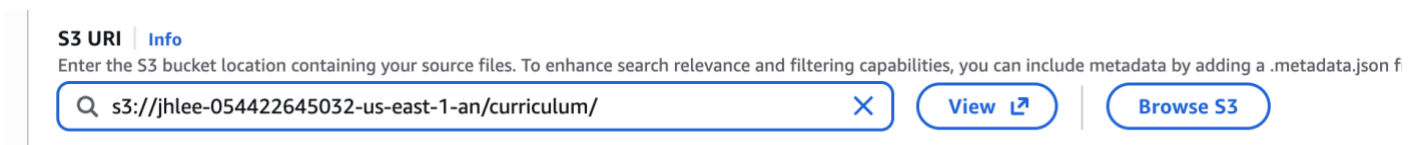
2. **Create** → **Knowledge Base with vector store** 를 선택합니다.



- 이름을 입력합니다. 예: university-curriculum-kb
- IAM permissions 에서 **Create and use a new service role** 를 선택합니다. (역할을 직접 만들 필요 없음 — 자동 생성)
- Data source 가 **Amazon S3** 로 선택돼 있는지 확인하고 **Next** 를 누릅니다.
- S3 URI 항목에서 **Browse S3** 를 눌러 모듈 2 의 버킷 → curriculum/ 폴더를 선택합니다.



폴더까지 반영됩니다.



7. **Parsing strategy** 단계에서 **Amazon Bedrock default parser** 를 선택합니다.

Parsing strategy | Info

The Amazon Bedrock Default parser handles common text formats, while specialized parsers are needed for other content types. Any remaining non-parseable content is processed according to the chosen embedding model's capabilities.

Amazon Bedrock default parser ☒

Best if only parsing text. This parser will ignore multimodal content. Commonly used with multimodal embedding model.

Data parsed:

- Text documents(e.g. Word, Excel, html, markdown, .txt, .csv)

Parser output:

- Extracted text

Amazon Bedrock Data Automation as parser ☐

Allows you to parse text and store multimodal content as text. Use this for high accuracy speech retrieval on audio/video files.

Data parsed:

- PDFs
- Images
- Audio
- Video

Parser output:

- Extracted text
- Descriptions for images, visuals, figures, charts
- Transcription for audio/video
- Video summarization

Foundation models as a parser ☐

Optimal for more advanced text and image parsing use cases. You can use the default parser prompt or customize it for your use case.

Data parsed:

- PDFs
- Images
- Structured/formatted text documents
- Visually rich documents

Parser output:

- Extracted text
- Descriptions for images, visuals, figures, charts

8. **Embeddings model** 로 **Titan Text Embeddings V2** 를 선택합니다.

9. **Vector store** 단계에서 **Quick create a new vector store** 를 선택하고, 아래에서 **Amazon S3 Vectors** 를 고릅니다.

10. **Next** 로 검토 화면을 확인한 뒤 **Create Knowledge Base** 를 누릅니다.

Amazon Bedrock > Knowledge Bases (KB) > Create Knowledge Base (KB) with vector store

API keys

Labs

settings Quotas [New](#)

compare Compare [New](#)

models

Creation of Knowledge Base — 'university-curriculum-kb' is in progress.

Creating role AmazonBedrockExecutionRoleForKnowledgeBase_azof0 in progress.

11. 상태가 **Available** 이 될 때까지 기다립니다. (수 분 소요)

✓ **체크포인트**

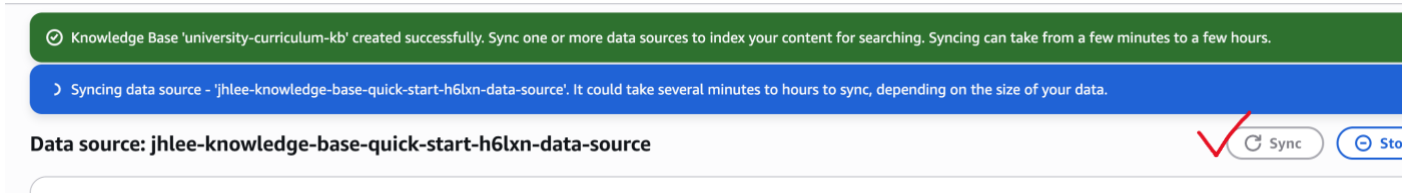
- ☐ Knowledge Base 상태 = Available
- ☐ 서비스 역할이 자동 생성됨
(AmazonBedrockExecutionRoleForKnowledgeBase_XXXXX)

모듈 4. 데이터 동기화 & 지식봇 테스트 (30 분)

4-1. 문서 색인(Sync)

Knowledge Base 를 만들었어도, 문서를 **동기화(Sync)**해야 벡터로 변환되어 검색 가능해집니다.

1. 방금 만든 Knowledge Base 상세 화면으로 들어갑니다.
2. **Data source** 섹션에서 데이터 소스를 선택하고 **Sync** 버튼을 누릅니다.



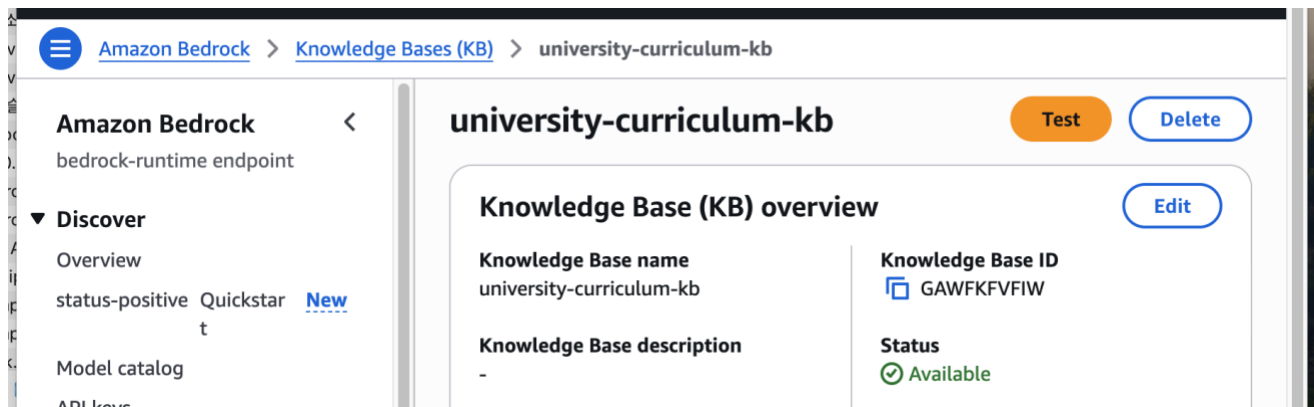
3. 상태가 **Syncing** → **Ready / Completed** 로 바뀔 때까지 기다립니다. (문서 3 개면 1~2 분)

💡 문서를 바꾸면

나중에 S3 의 문서를 추가·수정한 뒤 **Sync** 만 다시 누르면 지식봇이 최신 내용으로 갱신됩니다. 모델 재학습이 전혀 필요 없습니다 — 이것이 RAG 의 가장 큰 장점입니다.

4-2. 콘솔에서 질의응답 테스트

1. Knowledge Base 상세 화면 오른쪽의 **Test Knowledge Base** 패널을 엽니다.



2. **Select model** 을 눌러 **Claude Haiku 4.5** 을 선택합니다.
3. **Generate responses** 토글이 **켜짐** 상태인지 확인합니다. (꺼져 있으면 검색 결과만 보여줌)

▼ Retrieval and response generation

- ☐ **Agentic retrieval with answer generation - new**
Not supported for this KB type. Agentic retrieval with answer generation is only available for Managed Knowledge Bases.
- ☐ **Agentic retrieval only - new**
Not supported for this KB type. Agentic retrieval is only available for Managed Knowledge Bases.
- ☐ **Standard retrieval only**
Retrieve relevant content directly from a single KB.
- ☒ **Standard retrieval with answer generation**
Query the data sources and generate responses based on the retrieved results by using the specified foundation model.

Model

Choose a model for query planning and response generation.



Claude Haiku 4.5 v1 ⓘ ✎
Global Anthropic Claude Haiku 4.5

4. 아래 질문들을 차례로 입력해 답을 확인합니다.

테스트 질문	기대하는 동작
데이터 엔지니어링 과목의 선수과목은 무엇인가요?	"클라우드 컴퓨팅과 빅데이터 필수"를 근거와 함께 답변
클라우드 컴퓨팅 과목의 평가 비중을 알려줘	중간 25 / 기말 25 / 실습 30 / 팀플 20 답변
AI 기초에서 Transformer 는 몇 주차에 배우나요?	11 주차로 답변
세 과목 중 팀프로젝트 비중이 가장 높은 과목은?	데이터 엔지니어링(25%)으로 비교·답변
요리 레시피 추천해줘	문서에 없는 내용 → 모른다고 답해야 정상

5. 각 답변 아래의 **Details** 를 펼쳐, 답변이 어느 문서를 근거로 했는지 확인합니다.

✓ 체크포인트

- ☐ 문서 기반 질문에 정확히 답변함
- ☐ 출처(Citation)에 업로드한 .txt 파일이 표시됨
- ☐ 문서에 없는 질문에는 "모른다"고 답함 (환각 억제 확인)

모듈 5. 심화 실험 (20 분)

아래 실험으로 RAG 의 동작 원리를 더 깊이 체감합니다. 각 항목은 독립적이지 관심 가는 것부터 해도 됩니다.

실험 1. 검색만 vs 생성까지

Test 패널에서 **Generate responses** 토글을 **끄고** 같은 질문을 다시 해봅니다. 이번엔 LLM 답변 대신 **검색된 원문 청크**만 나옵니다. "검색(Retrieval)"과 "생성(Generation)"이 분리된 단계임을 눈으로 확인할 수 있습니다.

실험 2. 생성 모델 바꿔보기

Claude 4.6 Sonnet 으로 같은 질문을 던져 **답변 품질과 속도**를 비교합니다. Haiku 는 빠르고 저렴, Sonnet 은 더 정교합니다.

실험 4. 청킹 전략 비교 (새 KB 필요)

새 Knowledge Base 를 만들 때 **Chunking strategy** 를 **Fixed-size** 대신 **Hierarchical** 로 설정해, 긴 문서에서 검색 정확도가 어떻게 달라지는지 관찰합니다.

Knowledge Bases (KB) > Create Knowledge Base (KB) with vector store

Parsing strategy | Info
The Amazon Bedrock Default parser handles common text formats, while specialized parsers are needed for other content types. Any remaining non-parsed capabilities.

Amazon Bedrock default parser

Best if only parsing text. This parser will ignore multimodal content. Commonly used with multimodal embedding model.

Data parsed:

- Text documents(e.g. Word, Excel, html, markdown, .txt, .csv)

Parser output:

- Extracted text

Amazon Bedrock Data Automation as parser

Allows you to parse text and store multimodal content as text. Use this for high accuracy speech retrieval on audio/video files.

Data parsed:

- PDFs
- Images
- Audio
- Video

Default chunking
Automatically splits text into chunks of about 300 tokens in size, by default. If a document is less than or already 300 tokens, it's not split any further.

Fixed-size chunk Fixed-size chunking
Splits text into your set approximate token size.

Hierarchical chunking ✓
Organizes text chunks (nodes) into hierarchical structures of parent-child relationships. Each child node includes a reference to its parent node.

Semantic chunking
Organizes text chunks or groups of sentences by how semantically similar they are to each other.

No chunking
Suitable for documents that are already pre-processed or text split into separate files without any further chunking necessary.

Hierarchical chunking ▲
Organizes text chunks (nodes) into hierarchical structures of parent-child relationships. Each child node includes a reference to its parent n

청킹 방식	작동 원리 및 특징	추천 시나리오
Default chunking (기본 청킹)	문서를 기본적으로 약 300개의 토큰(Token) 크기로 자동 분할합니다. 만약 전체 문서가 300토큰 미만이라면 더 이상 쪼개지 않습니다.	가장 무난하고 빠른 기본 설정을 원할 때
Fixed-size chunking (고정 크기 청킹)	사용자가 **원하는 대략적인 토큰 크기(Token Size)**를 직접 지정하면, 시스템이 그 크기에 맞춰 글을 기계적으로 균등하게 등분합니다.	데이터의 길이와 쪼개지는 규격을 일정하게 통제하고 싶을 때
Hierarchical chunking (계층적 청킹) <i>(현재 이미지 선택됨)</i>	텍스트 덩어리들을 부모-자식(Parent-Child) 관계의 계층 구조 로 조직화합니다. 각 자식 노드는 부모 노드에 대한 참조 정보를 포함합니다.	세부적인 내용을 검색하면서도, 문맥(Context)의 거시적인 흐름을 놓치고 싶지 않을 때
Semantic chunking (의미론적 청킹)	글자 수나 토큰 수로 무조건 자르는 것이 아니라, 문장들이 서로 얼마나 의미상 유사한지(Semantically Similar) 분석하여 토픽이 바뀔 때 청크를 나눕니다.	계약서, 법률 문서, 논문 등 문맥과 의미적 연결성이 매우 중요한 고품질 문서를 다룰 때

💡 생각해볼 질문

- 문서에 없는 질문에 봇이 그럴듯한 거짓말을 한다면, 어떤 설정을 바꿔야 할까?
- 사내 규정이 매달 바뀐다면, RAG 와 Fine-tuning 중 무엇이 유리할까?

· **Fine-tuning(미세 조정)의 한계:** * 모델의 뇌(가중치 데이터) 자체를 새로 학습시키는 방식입니다. 규정이 바뀔 때마다 매달 막대한 컴퓨팅 비용과 시간을 들여 재학습을 시켜야 하므로 비용과 리소스 소모가 너무 큽니다.

· RAG(검색 증강 생성)의 강점:

- 모델은 그대로 두고, 연동된 데이터베이스(S3 등)의 문서 파일만 최신본으로 교체(Upsert)해주면 됩니다. 즉, 모델을 다시 학습시킬 필요 없이 실시간으로 지식을 업데이트할 수 있으므로, 정보가 자주 바뀌는 사내 규정 관리에는 RAG 가 가장 비용 효율적이고 정확합니다.

모듈 6. 리소스 정리

사용하지 않는 리소스를 정리합니다^^

삭제 순서

1. **Knowledge Base 삭제** — Bedrock → Knowledge Bases → 해당 KB 선택 → Delete. 삭제 안내에 적힌 연결된 벡터 스토어 이름을 메모해 둡니다.
2. **벡터 스토어 삭제** — OpenSearch Serverless 를 썼다면 OpenSearch → Serverless → Collections 에서 자동 생성된 컬렉션을 삭제합니다. S3 Vectors 를 썼다면 S3 → Vector buckets 에서 삭제합니다.
3. **S3 버킷 비우기 & 삭제** — S3 → 버킷 선택 → **Empty** 로 내용을 비운 뒤 **Delete** 로 버킷을 삭제합니다.

✓ 최종 체크포인트

- ☐ Knowledge Base 삭제됨
- ☐ 벡터 스토어(컬렉션 / 벡터 버킷) 삭제됨
- ☐ S3 버킷 비우고 삭제됨

부록 A. 트러블슈팅

콘솔 방식에서는 권한 오류가 거의 없지만, 그래도 막히면 아래 표를 참고하세요.

증상 / 메시지	원인	해결
임베딩 모델이 목록에 안 보임	모델 액세스 미활성화 또는 리전 불일치	모듈 1 재확인, 리전 us-east-1
AccessDenied / 권한 오류	기존 역할을 선택함	KB 생성 시 "Create and use a new service role" 선택
Bucket already exists	버킷 이름 중복	이름 뒤에 숫자·날짜 추가
Sync 후에도 답을 못 찾음	동기화 미완료 또는 빈 폴더	Sync 상태 Completed 확인, S3 경로 재확인
문서에 있는데도 "모른다"	검색 청크 수가 너무 적음	Configurations 에서 chunk 수 늘리기
영동한 답(환각)	Generate 토크 커짐 + 근거 부족	출처 확인, 질문을 더 구체적으로

왜 노트북(코드) 방식은 더 어려웠나

참고로, 이전 실습에서 겪은 오류들은 모두 "사전 환경을 손으로 만들어야 해서" 생긴 것이었습니다. 콘솔 Quick create 는 이 단계를 자동화합니다.

코드 방식에서 났던 오류	콘솔 방식에서는
s3:PutObject AccessDenied	콘솔 업로드라 권한 불필요
unable to assume the given role	서비스 역할 자동 생성
403 Forbidden (OpenSearch 접근)	벡터 스토어·액세스 정책 자동 구성
aoss:CreateSecurityPolicy 거부 등	사용자가 권한을 만질 필요 없음

부록 B. Azure ↔ AWS 서비스 대응표

Azure 경험이 있다면 아래 대응을 떠올리면 빠르게 이해됩니다.

개념	Azure	AWS (이 실습)
관리형 LLM 서비스	Azure OpenAI Service	Amazon Bedrock
파일 저장소	Blob Storage	Amazon S3
벡터 검색/DB	Azure AI Search	OpenSearch Serverless / S3 Vectors
RAG 통합 기능	Azure OpenAI On Your Data	Bedrock Knowledge Bases
임베딩 모델	text-embedding-3	Amazon Titan Text Embeddings V2
생성 모델	GPT-4o	Anthropic Claude 3.5 Sonnet
권한 관리	Entra ID / RBAC	IAM (역할·정책)
노트북 환경	Azure ML Studio	Amazon SageMaker

💡 핵심 차이

Azure의 “On Your Data”처럼, AWS의 **Bedrock Knowledge Bases**도 문서를 올리면 RAG 파이프라인을 거의 자동으로 구성해 줍니다. 가장 큰 차이는 **AWS는 권한(IAM)을 더 명시적으로 다룬다**는 점인데, 콘솔 Quick create가 그 부분을 대신 처리해 줍니다.

부록 C. 전체 완료 체크리스트

실습 중

- ☐ 리전이 us-east-1 로 설정됨
- ☐ Titan Text Embeddings V2 액세스 = granted
- ☐ Claude 모델 액세스 = granted
- ☐ S3 버킷 생성 및 문서 3 개 업로드
- ☐ Knowledge Base 생성 (새 서비스 역할 자동 생성)
- ☐ 데이터 소스 Sync 완료
- ☐ 콘솔 Test 에서 문서 기반 질의응답 성공
- ☐ 출처(Citation) 확인

실습 후 (비용 방지)

- ☐ Knowledge Base 삭제
- ☐ 벡터 스토어 삭제
- ☐ S3 버킷 비우고 삭제

부록 D. 제공 샘플 문서

- ai_basic.txt — 인공지능 기초 (15 주 강의계획, 평가기준)
- cloud_computing.txt — 클라우드 컴퓨팅과 빅데이터
- data_engineering.txt — 데이터 엔지니어링 실무

세 파일은 가이드와 같은 폴더의 sample-docs 안에 있습니다.